Software Testing & Quality Report

SmartCart – Full Stack E-Commerce Shopping Website

Role: Software Test Engineer
Date: July 2026
Methodology: Functional Manual Testing

1. Project Information

Project Name:	SmartCart – Full Stack E-Commerce Shopping Website
Frontend Tech:	React.js (v16+), HTML5, CSS3, JavaScript (ES6+), Axios, React Router v5
Backend Tech:	Java 21, Spring Boot 3.3, Spring Data JPA, Maven
Database:	MySQL Server 8.0+
Tested By:	Behara Karthik (Software Test Engineer)

2. Testing Environment

The manual functional verification was performed under the following local development environment specifications:

- **Operating System:** Windows 11 (64-bit)
- **Browser:** Google Chrome (v126+) / Microsoft Edge
- **Backend Application Server:** Embedded Apache Tomcat on port `8080` (Spring Boot 3.3 / Java 21)
- **Frontend Web Server:** Node.js / React Development Server on port `3000`
- **Database Host:** MySQL Server on `localhost:3306`
- **API Testing Client:** Postman & Chrome DevTools Network Tab

3. Testing Methodology

The primary approach utilized for this evaluation is **Black-Box Manual Functional Testing**. Test suites were designed based on end-user functional requirements, validating client-side UI

workflows, input validation, context state persistence, RESTful API JSON payload transmission via Axios, Spring Boot business logic execution, and database CRUD transaction consistency.

4. Functional Test Cases

ID	Feature	Expected Result	Actual Result	Status
TC-001	User Registration	Submit user credentials; save user entity to MySQL database.	User registered successfully; record created in user table.	PASS
TC-002	User Login	Authenticate user against database; persist user state in localStorage.	Login successful; user session restored on refresh.	PASS
TC-003	Product Catalog Display	Fetch products via GET /api/products/all; render cards.	Products rendered with name, price, image & dynamic badge.	PASS
TC-004	Category Filter	Filter products by selecting specific category pills.	Catalog displays only products matching category ID.	PASS
TC-005	Real-Time Search	Filter product grid instantly based on navbar search query.	Product list filters dynamically in real-time.	PASS
TC-006	Price Sorting	Sort products by Price (Low to High / High to Low).	Products re-order accurately according to price.	PASS
TC-007	Add to Shopping Cart	Click "Add to Cart"; increment item count badge in navbar.	Item added to cart state; navbar badge counter updates.	PASS
TC-008	Cart Drawer Management	Modify item quantity (+/-) or remove item in cart modal.	Item quantities update and total recalculates live.	PASS
TC-009	Checkout & Order Creation	Submit shipping details & payment method (POST /api/orders/create).	Order created in database with status PENDING; receipt shown.	PASS
TC-010	Customer Order History	Navigate to /orders; retrieve user order list.	Past orders displayed with status badges, items & receipt.	PASS
TC-011	Admin Order Status Update	Admin modifies order status dropdown (PUT	Status updated to SHIPPED/DELIVERED in	PASS

ID	Feature	Expected Result	Actual Result	Status
		/api/orders/{id}/status).	DB & UI.	
TC-012	REST API & DB Integration	React Axios calls Spring Boot Controllers & executes JPA queries.	All endpoints return 200 OK with formatted JSON payloads.	PASS

5. Test Execution Summary

Total Test Cases Executed: 12	**Passed:** 12 (100%)
Failed: 0 (0%)	**Blockers / Critical Defect Count:** 0
Test Coverage: 100% Functional Flow	**Overall Status:** PASSED / VERIFIED

6. Conclusion

The manual functional verification of the **SmartCart – Full Stack E-Commerce Shopping Website** was completed successfully. All core functional modules—including user authentication, catalog navigation, real-time search, cart drawer state management, multi-step checkout, customer order tracking, and admin order management—demonstrate robust stability, accurate data persistence in MySQL, and reliable RESTful API communication between React and Spring Boot. The system meets all functional criteria and is recommended for deployment and academic submission.

7. Test Execution Proof & Screenshot Placeholders

[Screenshot 1: Backend Server Running]

Spring Boot application started on port 8080

[Screenshot 2: Frontend Running]

React dev server running on http://localhost:3000

[Screenshot 3: Home Page & Hero Banner]

[Screenshot 4: Login & Authentication]

User login page view

SmartCart landing page view

[Screenshot 5: Product Catalog & Search]

Products page with live search & sorting

[Screenshot 6: Shopping Cart Drawer]

Cart modal with items & subtotal calculation

[Screenshot 7: MySQL Database Connection & Tables]

MySQL Workbench view displaying ecommerce database tables